

# Graphics Processing Unit

Mathieu Léonardon



**IMT Atlantique**  
Bretagne-Pays de la Loire  
École Mines-Télécom

This course is mostly based on the course of Jukka Suomela from the Aalto university (<https://ppc.cs.aalto.fi/>), which is available under [CC BY 4.0](#) license.  
This course is also under [CC BY 4.0](#).



# Plan

- 1 GPU programming
- 2 GPU programming with CUDA
- 3 Sample application: shortcut problem
- 4 Memory access patterns in CUDA programs
- 5 Warps, blocks, and shared memory
- 6 How does the GPU execute code?
- 7 GPU programming — conclusions

# Plan

- 1 GPU programming
- 2 GPU programming with CUDA
- 3 Sample application: shortcut problem
- 4 Memory access patterns in CUDA programs
- 5 Warps, blocks, and shared memory
- 6 How does the GPU execute code?
- 7 GPU programming — conclusions

# GPU – graphics processing unit

- All modern computers have at least two processors:
  - **CPU**: what we have been using so far
  - **GPU**: lots of more computing power available and we can use it, too!
- **CPU** in our classroom computers [Intel Xeon E3-1230v5, “Skylake”]:
  - **460 billion** single-precision floating-point operations / second
  - **230 billion** double-precision floating-point operations / second
- **GPU** in our classroom computers [NVIDIA Quadro K2200, “Maxwell”]:
  - **1400 billion** single-precision floating-point operations / second
  - **45 billion** double-precision floating-point operations / second

# GPU - graphics processing unit

- All modern computers have at least two processors
  - **CPU**: what we have been using so far
  - **GPU**: lots of more computing power available and we can use it, too!
- **CPU** that we will use : **Intel(R) Core(TM) i5-7400 CPU @ 3.00GHz**
  - Vector Floating point math : **96 GFLOPS**
- **GPU** that we will use : **NVIDIA GeForce GTX 1060 6 GB**
  - Parallel Floating point math : **1920 GFLOPS**

# GPU - graphics processing unit

- All modern computers have at least two processors
  - **CPU**: what we have been using so far
  - **GPU**: lots of more computing power available and we can use it, too!
- **CPU** that we will use : **Intel(R) Core(TM) i5-7400 CPU @ 3.00GHz**
  - Vector Floating point math : **96 GFLOPS**
- **GPU** that we will use : **NVIDIA GeForce GTX 1060 6 GB**
  - Parallel Floating point math : **1920 GFLOPS**

# GPU - graphics processing unit

- All modern computers have at least two processors
  - **CPU**: what we have been using so far
  - **GPU**: lots of more computing power available and we can use it, too!
- **CPU** that we will use : **Intel(R) Core(TM) i5-7400 CPU @ 3.00GHz**
  - Vector Floating point math : **96 GFLOPS**
- **GPU** that we will use : **NVIDIA GeForce GTX 1060 6 GB**
  - Parallel Floating point math : **1920 GFLOPS**

# Common feature: lots of wide, pipelined arithmetic units

## CPU

- Pipelined arithmetic units:  
1 new operation per cycle
- **Vector** operations:  
8 similar operations
- Lots of arithmetic units:  
4 vector operations  
in parallel e.g. for FMA

## GPU

- Pipelined arithmetic units:  
1 new operation per cycle
- **Warp of threads:**  
32 similar operations
- Lots of arithmetic units:  
 $5 \times 4$  warps executed  
in parallel e.g. for FMA



# Different tradeoffs between parallelism and clock frequency

## CPU

- **32** single-precision arithmetic operations per cycle
- **3.3** GHz (Turbo)
- FMA = **2** operations
- **211** billion arithmetic operations per second

## GPU

- **1280** single-precision arithmetic operations per cycle
- **1.5** GHz (Turbo)
- FMA = **2** operations
- **1920** billion arithmetic operations per second

# Differences in programming models

## CPU

- “**SIMD**” — single instruction, multiple data
- **One thread per core**

## GPU

- “**SIMT**” — single instruction, multiple threads
- **Very many threads**
  - threads organized in “blocks”
  - blocks consist of “warps”
  - **warp = 32 threads**  
**always works together**

# Differences in programming models

## CPU

- “**SIMD**” — single instruction, multiple data
- **One thread per core**
  - each thread refers to **vector registers**
  - each thread performs **vector operations**

## GPU

- “**SIMT**” — single instruction, multiple threads
- **Very many threads**
  - each thread refers to **scalar registers**
  - each thread performs **scalar operations**

# Differences in programming models

## CPU

- “**SIMD**” — single instruction, multiple data
- **One thread** says e.g.:
  - compute vector sum  
 $z[i] = x[i] + y[i]$   
for all  $i = 0 \dots 7$

8 similar  
operations  
in parallel

## GPU

- “**SIMT**” — single instruction, multiple threads
- **All threads of a warp** say e.g.:
  - compute scalar sum  
 $z = x + y$

32 similar  
operations  
in parallel

# Differences in programming models

## CPU

- “**SIMD**” — single instruction, multiple data
- **One thread** says e.g.:
  - read one vector from memory and store it in vector variable x
  - can read 8 scalars in one step, **have to be in consecutive memory locations**

## GPU

- “**SIMT**” — single instruction, multiple threads
- **All threads of a warp** say e.g.:
  - read one scalar from memory and store it in scalar variable x
  - can read 32 scalars in one step, **do not need to be consecutive memory locations**

# Differences in programming models

## CPU

- “**SIMD**” — single instruction, multiple data
- Some SIMT solutions are hard to implement here

## GPU

- “**SIMT**” — single instruction, multiple thread
- Any SIMD solution easy to implement here (?)

# Forms of parallelism

## CPU

- Threads
- Vector operations

## GPU

- **Threads — *lots of them!***
- ~~Vector operations~~

# Forms of parallelism

## CPU

- Threads
- Vector operations
- Instruction-level parallelism

## GPU

- **Threads — *lots of them!***
- ~~Vector operations~~
- ***Instruction-level parallelism??***



# How the hardware tries to keep pipelines busy

## CPU

- Looks at the instruction stream ***far into the future***
  - tries to find some **instruction** that is ready for execution
  - **instruction-level parallelism important**
  - small number of threads enough

## GPU

- Only looks at the ***first instruction*** of each warp
  - tries to find some **warp** that is ready for execution
  - instruction-level parallelism not that important
  - **large number of warps needed**

# Forms of parallelism

## CPU

- Threads
- Vector operations
- Instruction-level parallelism

## GPU

- **Threads — *lots of them!***
- ~~Vector operations~~
- ~~Instruction level parallelism~~

# Different history, different hardware design

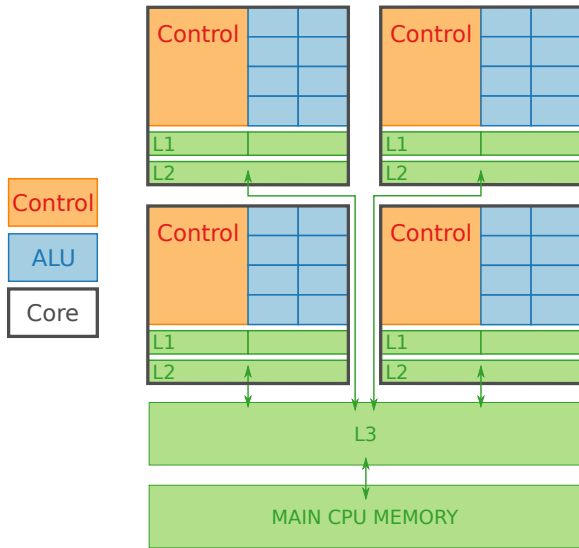
## CPU

- Tries to run old sequential code reasonably well
- Lots of complicated hardware to support that
  - out-of-order execution
  - high clock frequency
  - many layers of cache

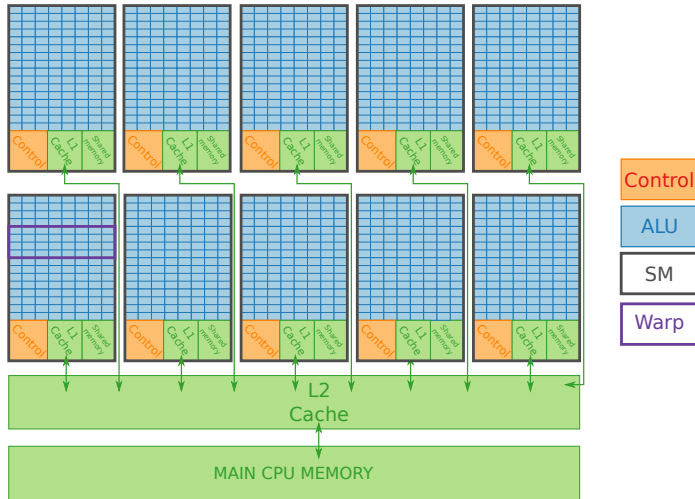
## GPU

- Does not care anything about running old sequential code
- Simpler hardware
  - transistors used for arithmetic, not for control logic
  - easier to add more parallel units than increase clock speed

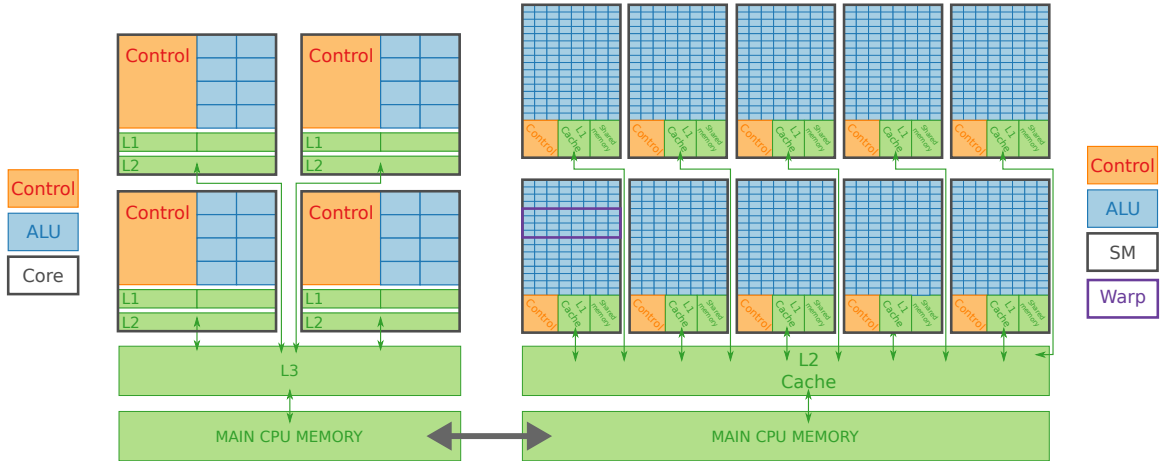
# CPU schematic



# GPU schematic



# CPU <=> GPU schematic



# Plan

- 1 GPU programming
- 2 GPU programming with CUDA
- 3 Sample application: shortcut problem
- 4 Memory access patterns in CUDA programs
- 5 Warps, blocks, and shared memory
- 6 How does the GPU execute code?
- 7 GPU programming — conclusions

# GPU programming with CUDA

- We will use NVIDIA GPUs and **CUDA** programming environment
  - CUDA code  $\approx$  C++ code with some extensions
  - compile with **nvcc**, run as usual
- Just compiling your code with **nvcc** doesn't do anything yet
  - *your `main()` function still runs on the CPU!*
- In your program, you need to specify what the GPU should do
  - you define a so-called “**kernel**” function
  - *you explicitly ask the GPU to run the “kernel” with many threads!*



# CUDA basics

## What we would like to parallelize

```
for (int i = 0; i < 100; ++i) {  
    for (int j = 0; j < 128; ++j) {  
        foo(i, j);  
    }  
}
```

GPU should run  
these operations,  
preferably in parallel:

- foo(0, 0)
- foo(0, 1)
- foo(0, 2)
- foo(0, 3)
- ...
- foo(99, 127)

# CUDA basics

## Parallel GPU solution

GPU {

```
__global__ void mykernel() {  
    int i = blockIdx.x;  
    int j = threadIdx.x;  
    foo(i, j);  
}
```

Which thread am I?

CPU {

```
int main() {  
    mykernel<<<100, 128>>>();  
}
```

Create 100 blocks,  
each with 128 threads,  
and let them all run  
function mykernel!

GPU will run  
these operations,  
possibly in parallel:

- foo(0, 0)
- foo(0, 1)
- foo(0, 2)
- foo(0, 3)
- ...
- foo(99, 127)

# CUDA basics

## Parallel GPU solution

GPU {  
\_\_global\_\_ void mykernel() {  
 int i = blockIdx.x;  
 int j = threadIdx.x;  
 foo(i, j);  
}  
}

CPU {  
int main() {  
 mykernel<<<100, 128>>>();  
}

## Equivalent sequential code

```
int main() {  
    for (int i = 0; i < 100; ++i) {  
        for (int j = 0; j < 128; ++j) {  
            foo(i, j);  
        }  
    }  
}
```

# Example: split evenly

- What is the best way to **split**  $1^5, 2^5, 3^5, \dots, 30^5$  in **two parts** such that their **sums** are as close to each other as possible?

|        |        |        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|
| $1^5$  | $2^5$  | $3^5$  | $4^5$  | $5^5$  | $6^5$  | $7^5$  | $8^5$  | $9^5$  | $10^5$ |
| $11^5$ | $12^5$ | $13^5$ | $14^5$ | $15^5$ | $16^5$ | $17^5$ | $18^5$ | $19^5$ | $20^5$ |
| $21^5$ | $22^5$ | $23^5$ | $24^5$ | $25^5$ | $26^5$ | $27^5$ | $28^5$ | $29^5$ | $30^5$ |

# Example: split evenly

- What is the best way to **split**  $1^5, 2^5, 3^5, \dots, 30^5$  in **two parts** such that their **sums** are as close to each other as possible?

|        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|
| $1^5$  | $2^5$  | $4^5$  | $6^5$  | $10^5$ |        |
| $11^5$ | $12^5$ | $13^5$ | $15^5$ | $17^5$ | $19^5$ |
| $21^5$ | $22^5$ | $23^5$ | $24^5$ | $27^5$ | $30^5$ |

sum: 67 830 947

|        |        |        |        |       |  |
|--------|--------|--------|--------|-------|--|
| $3^5$  | $5^5$  | $7^5$  | $8^5$  | $9^5$ |  |
| $14^5$ | $16^5$ | $18^5$ | $20^5$ |       |  |
| $25^5$ | $26^5$ | $28^5$ | $29^5$ |       |  |

sum: 66 156 478

# Example: split evenly

- What is the best way to **split**  $1^5, 2^5, 3^5, \dots, 30^5$  in **two parts** such that their **sums** are as close to each other as possible?



# Example: split evenly

- What is the best way to **split**  $1^5, 2^5, 3^5, \dots, 30^5$  in **two parts** such that their **sums** are as close to each other as possible?
- We will solve this with a **naive brute force algorithm**
- First with **CPUs with a sequential program**
- Then with **GPUs with a massively parallel program**

# Example: split evenly

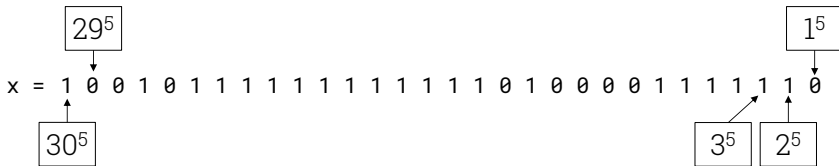
- What is the best way to **split**  $1^5, 2^5, 3^5, \dots, 30^5$  in **two parts** such that their **sums** are as close to each other as possible?
- Algorithms: just try out all  $2^{30}$  **cases** and see what is best



# Example: split evenly

Each case is represented as a **30-bit binary number  $x$**

Bit 0 in position  $i$ : number  $(i + 1)^5$  in the **first part**



Bit 1 in position  $i$ : number  $(i + 1)^5$  in the **second part**

```
inline int p5(int i) { return i * i * i * i * i; }
```

```
inline int value(int x) {  
    int a = 0;  
    for (int i = 0; i < 30; ++i) {  
        if (x & (1 << i)) {  
            a += p5(i+1);  
        } else {  
            a -= p5(i+1);  
        }  
    }  
    return abs(a);  
}
```

x = one way to split our numbers  
value(x) = absolute difference  
between the sum of the first part  
and the sum of the second part

```
inline int p5(int i) { return i * i * i * i * i; }
```

```
inline int value(int x) {  
    int a = 0;  
    for (int i = 0; i < 30; ++i) {  
        if (x & (1 << i)) {  
            a += p5(i+1);  
        } else {  
            a -= p5(i+1);  
        }  
    }  
    return abs(a);  
}
```

**Find  $0 \leq x < 2^{30}$   
that minimizes  
value(x)**

# Sequential CPU solution

```
constexpr int total = 1 << 30;
int best_x = 0;
int best_v = value(best_x);
for (int x = 0; x < total; ++x) {
    int v = value(x);
    if (v < best_v) {
        best_x = x;
        best_v = v;
    }
}
```

**Find  $0 \leq x < 2^{30}$   
that minimizes  
value(x)**

# GPU: splitting work

- We have got  $2^{30}$  cases to check
- How many **blocks** to create?
- How many **threads** per block?
- How many cases does one thread check?

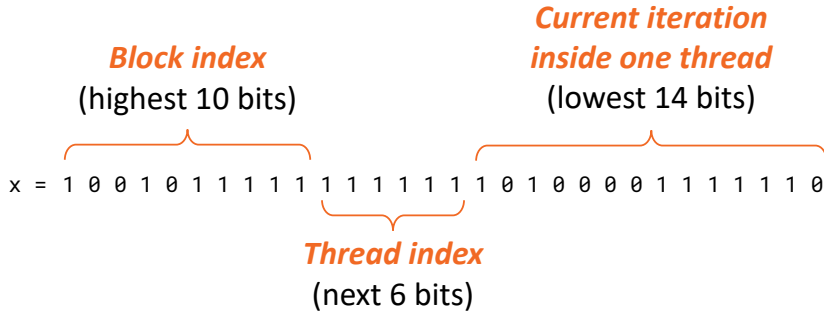
# GPU: splitting work

- We have got  $2^{30}$  cases to check
- How many **blocks** to create?
- How many **threads** per block?
- If we have e.g.  $2^{30}$  threads in total, each thread will only check **1** case
  - too little useful work per thread
  - too much overhead e.g. in launching kernel, communicating result

# GPU: splitting work

- We have got  $2^{30}$  cases to check
- **Blocks:**
  - we need to have lots of blocks ready for execution
  - our choice here:  $2^{10} = 1024$  blocks
- **Threads per block:**
  - reasonable block size is a multiple of one **warp** = 32 threads
  - our choice here:  $2^6 = 64$  threads
- Each thread will need to check  $2^{30} / (2^{10} \cdot 2^6) = 2^{14}$  cases

# GPU: splitting work



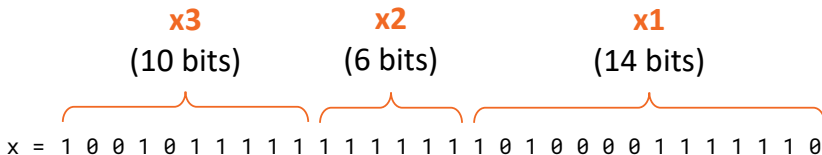


# GPU: coordination between threads

- Let's keep things as simple as possible
- Allocate one word of GPU memory per thread
- ***GPU: each thread will write its local optimum in GPU memory***
- Copy results from GPU memory to CPU memory
- ***CPU: find the best split among local optima***

```
__global__ void mykernel(int* r) {  
    int x3 = blockIdx.x;  
    int x2 = threadIdx.x;
```

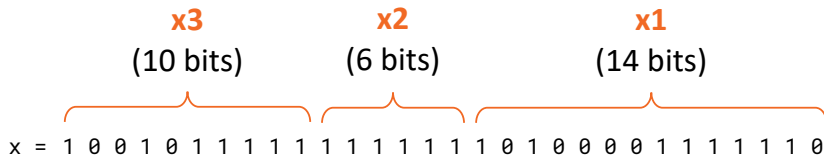
What is my part  
of search space?



```
}
```

```
__global__ void mykernel(int* r) {
    int x3 = blockIdx.x;
    int x2 = threadIdx.x;
```

What is my part  
of search space?



```
    r[(x3 << 6) | x2] = best_x;
}
```

Save best solution in my  
part of search space

```

__global__ void mykernel(int* r) {
    int x3 = blockIdx.x;
    int x2 = threadIdx.x;
    int best_x = 0;
    int best_v = value(best_x);
    for (int x1 = 0; x1 < iterations; ++x1) {
        int x = (x3 << 20) | (x2 << 14) | x1;
        int v = value(x);
        if (v < best_v) {
            best_x = x;
            best_v = v;
        }
    }
    r[(x3 << 6) | x2] = best_x;
}

```

What is my part  
of search space?

Mostly  
normal  
sequential  
C++ code  
here

Save best solution in my  
part of search space

```
constexpr int blocks = 1 << 10;
constexpr int threads = 1 << 6;

int* rGPU = NULL;
cudaMalloc((void*)&rGPU, blocks * threads * sizeof(int));

mykernel<<<blocks, threads>>>(rGPU);

std::vector<int> r(blocks * threads);
cudaMemcpy(r.data(), rGPU, blocks * threads * sizeof(int),
            cudaMemcpyDeviceToHost);
cudaFree(rGPU);

// Find x in r that
// minimizes value(x)
```

```
constexpr int blocks = 1 << 10;  
constexpr int threads = 1 << 6;
```

Allocate GPU  
memory for result

```
int* rGPU = NULL;  
cudaMalloc((void*)&rGPU, blocks * threads * sizeof(int));  
  
mykernel<<<blocks, threads>>>(rGPU);  
  
std::vector<int> r(blocks * threads);  
cudaMemcpy(r.data(), rGPU, blocks * threads * sizeof(int),  
            cudaMemcpyDeviceToHost);  
cudaFree(rGPU);  
  
// Find x in r that  
// minimizes value(x)
```

```
constexpr int blocks = 1 << 10;  
constexpr int threads = 1 << 6;
```

Allocate GPU  
memory for result

```
int* rGPU = NULL;  
cudaMalloc((void*)&rGPU, blocks * threads * sizeof(int));
```

```
mykernel<<<blocks, threads>>>(rGPU);
```

Launch  $2^{16}$   
threads on GPU

```
std::vector<int> r(blocks * threads);  
cudaMemcpy(r.data(), rGPU, blocks * threads * sizeof(int),  
           cudaMemcpyDeviceToHost);  
cudaFree(rGPU);
```

```
// Find x in r that  
// minimizes value(x)
```

```
constexpr int blocks = 1 << 10;  
constexpr int threads = 1 << 6;
```

**Allocate GPU  
memory for result**

```
int* rGPU = NULL;  
cudaMalloc((void*)&rGPU, blocks * threads * sizeof(int));
```

```
mykernel<<<blocks, threads>>>(rGPU);
```

**Launch  $2^{16}$   
threads on GPU**

```
std::vector<int> r(blocks * threads);  
cudaMemcpy(r.data(), rGPU, blocks * threads * sizeof(int),  
            cudaMemcpyDeviceToHost);  
cudaFree(rGPU);
```

**Copy result back from GPU  
memory to CPU memory**

```
// Find x in r that  
// minimizes value(x)
```



```
constexpr int blocks = 1 << 10;  
constexpr int threads = 1 << 6;
```

**Allocate GPU  
memory for result**

```
int* rGPU = NULL;  
cudaMalloc((void*)&rGPU, blocks * threads * sizeof(int));
```

```
mykernel<<<blocks, threads>>>(rGPU);
```

**Launch  $2^{16}$   
threads on GPU**

```
std::vector<int> r(blocks * threads);  
cudaMemcpy(r.data(), rGPU, blocks * threads * sizeof(int),  
            cudaMemcpyDeviceToHost);
```

```
cudaFree(rGPU);
```

**Free memory**

**Copy result back from GPU  
memory to CPU memory**

```
// Find x in r that  
// minimizes value(x)
```

```
constexpr int blocks = 1 << 10;
constexpr int threads = 1 << 6;

int* rGPU = NULL;
cudaMalloc((void*)&rGPU, blocks * threads * sizeof(int));

mykernel<<<blocks, threads>>>(rGPU);

std::vector<int> r(blocks * threads);
cudaMemcpy(r.data(), rGPU, blocks * threads * sizeof(int),
            cudaMemcpyDeviceToHost);
cudaFree(rGPU);

// Find x in r that
// minimizes value(x)
```

**Now vector "r" contains  
the best result for each thread,  
just check which of these is  
the global optimum**

# Try it out

- Compile & link with “**nvcc**” instead of “**g++**”
- Run as usual
  - sequential CPU solution: **38 seconds**
  - parallel GPU solution: **0.3 seconds**

$$1^5 + 2^5 + 3^5 + 4^5 + 5^5 + 9^5 + 10^5 + 12^5 + 15^5 + 16^5 + 17^5 + 19^5 + 22^5 + 23^5 + 24^5 + 25^5 + 27^5 + 28^5 = 66\,993\,712$$

$$6^5 + 7^5 + 8^5 + 11^5 + 13^5 + 14^5 + 18^5 + 20^5 + 21^5 + 26^5 + 29^5 + 30^5 = 66\,993\,713$$

```
constexpr int blocks = 1 << 10;
constexpr int threads = 1 << 6;

int* rGPU = NULL;
cudaMalloc((void*)&rGPU, blocks * threads * sizeof(int));

mykernel<<<blocks, threads>>>(rGPU);

std::vector<int> r(blocks * threads);
cudaMemcpy(r.data(), rGPU, blocks * threads * sizeof(int),
            cudaMemcpyDeviceToHost);
cudaFree(rGPU);

// Find x in r that
// minimizes value(x)
```

**Error checking  
omitted!**

# Typical program structure

- **GPU side:**

- “**kernel**” that does one small part of work

- **CPU side:**

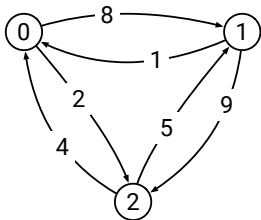
- do pre-processing if needed
- allocate GPU memory for input & output
- copy input from CPU memory to GPU memory
- **launch kernel** (lots of blocks, lots of threads)
- copy result back from GPU memory to CPU memory
- release GPU memory
- do post-processing if needed

# Plan

- 1 GPU programming
- 2 GPU programming with CUDA
- 3 Sample application: shortcut problem**
- 4 Memory access patterns in CUDA programs
- 5 Warps, blocks, and shared memory
- 6 How does the GPU execute code?
- 7 GPU programming — conclusions

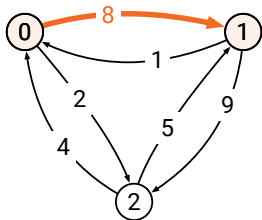
# Sample application: cheapest 2-hop path

d (input):



# Sample application: cheapest 2-hop path

d (input):

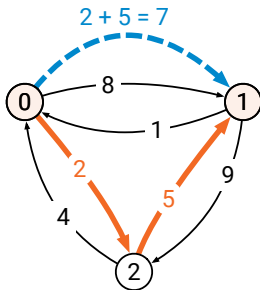
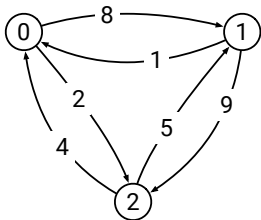


**Cost of traveling  
directly  $0 \rightarrow 1$**



# Sample application: cheapest 2-hop path

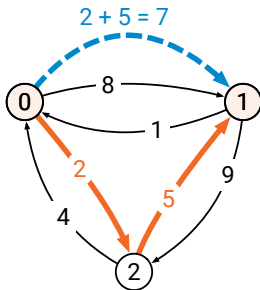
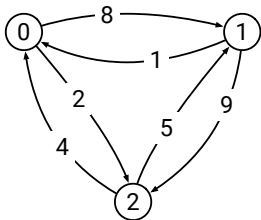
d (input):



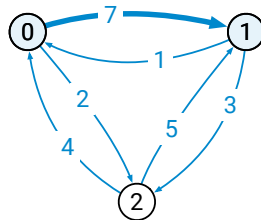
**Cost of traveling**  
 $0 \rightarrow 2 \rightarrow 1$

# Sample application: cheapest 2-hop path

d (input):

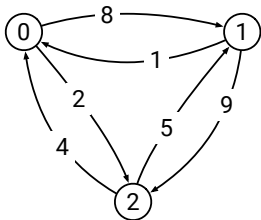


r (output):

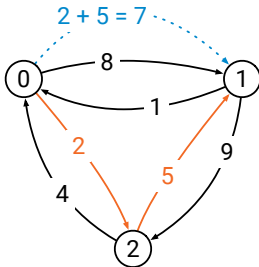


# Sample application: cheapest 2-hop path

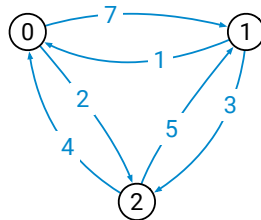
d (input):



```
d[] = { 0, 8, 2,  
        1, 0, 9,  
        4, 5, 0 }
```



r (output):

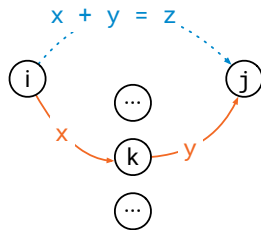


```
r[] = { 0, 7, 2,  
        1, 0, 3,  
        4, 5, 0 }
```

```

void step(float* r, const float* d, int n) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            float v = infinity;
            for (int k = 0; k < n; ++k) {
                float x = d[n*i + k];
                float y = d[n*k + j];
                float z = x + y;
                v = min(v, z);
            }
            r[n*i + j] = v;
        }
    }
}

```

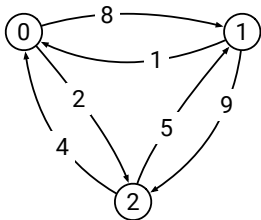


# Plan

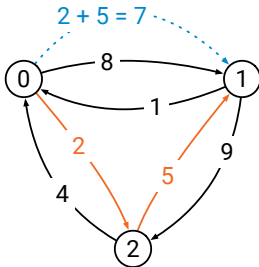
- 1 GPU programming
- 2 GPU programming with CUDA
- 3 Sample application: shortcut problem
- 4 Memory access patterns in CUDA programs**
- 5 Warps, blocks, and shared memory
- 6 How does the GPU execute code?
- 7 GPU programming — conclusions

# Sample application: cheapest 2-hop path

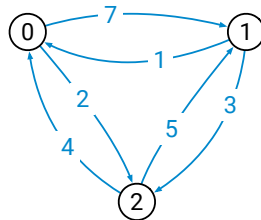
d (input):



```
d[] = { 0, 8, 2,  
        1, 0, 9,  
        4, 5, 0 }
```



r (output):

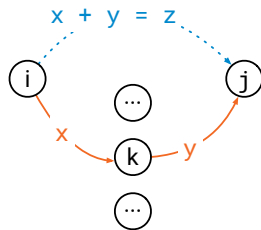


```
r[] = { 0, 7, 2,  
        1, 0, 3,  
        4, 5, 0 }
```

```

void step(float* r, const float* d, int n) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            float v = infinity;
            for (int k = 0; k < n; ++k) {
                float x = d[n*i + k];
                float y = d[n*k + j];
                float z = x + y;
                v = min(v, z);
            }
            r[n*i + j] = v;
        }
    }
}

```



# Splitting work

- Work to do:
  - need to compute  **$n \times n$  results**
  - computing one result takes  $n$  steps
- How do we split this in blocks and threads?
- Natural idea:
  - one thread computes **one result**
  - one block computes  **$b \times b$  results**, for some suitable  $b$
  - if we choose e.g.  $b = 16$ , then a block consists of 8 warps



# Splitting work

- Example: input dimensions are 1600 x 1600:
  - we want to create **100 x 100 blocks**
  - each block consists of **16 x 16 threads**
- Create 10 000 blocks with 256 threads:
  - blocks numbered **0 ... 9999**
  - threads numbered **0 ... 255**
- Convert block & thread index to  $(i, j)$  pair:
  - thread number **123** in block number **4567**  
computes the result for  $i = ???$  and  $j = ???$

# Splitting work

- Example: input dimensions are 1600 x 1600:
  - we want to create **100 x 100 blocks**
  - each block consists of **16 x 16 threads**
- Create 10 000 blocks with 256 threads:
  - blocks numbered **0 ... 9999**
  - threads numbered **0 ... 255**
- Convert block & thread index to  $(i, j)$  pair:
  - thread number **123** = **7** · 16 + **11** in block number **4567** = **45** · 100 + **67**  
computes the result for  $i = \mathbf{67} \cdot 16 + \mathbf{11}$  and  $j = \mathbf{45} \cdot 16 + \mathbf{7}$

# Splitting work: using 2D indexes

- Example: input dimensions are 1600 x 1600:
  - we want to create **100 x 100 blocks**
  - each block consists of **16 x 16 threads**
- Create 10 000 blocks with 256 threads using 2D indexes:
  - blocks numbered **(0, 0) ... (99, 99)**
  - threads numbered **(0, 0) ... (15, 15)**
- Convert block & thread coordinates to  $(i, j)$  pair:
  - thread number **(11, 7)** in block number **(67, 45)**  
computes the result for  $i = 67 \cdot 16 + 11$  and  $j = 45 \cdot 16 + 7$

# Splitting work: rounding

- Example: input dimensions are 1601 x 1601:
  - we want to create **101 x 101 blocks**
  - each block consists of **16 x 16 threads**
- Create 10 000 blocks with 256 threads using 2D indexes:
  - blocks numbered **(0, 0) ... (100, 100)**
  - threads numbered **(0, 0) ... (15, 15)**
- There will be some threads with  $i \geq 1601$  and/or  $j \geq 1601$ , they will do nothing

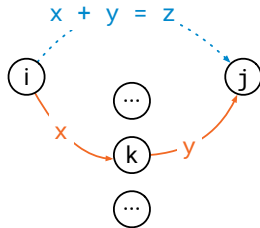
```

__global__ void mykernel(float* r, const float* d, int n) {
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    int j = threadIdx.y + blockIdx.y * blockDim.y;
    if (i >= n || j >= n)
        return;
    float v = HUGE_VALF;
    for (int k = 0; k < n; ++k) {
        float x = d[n*i + k];
        float y = d[n*k + j];
        float z = x + y;
        v = min(v, z);
    }
    r[n*i + j] = v;
}

```

What if n is not a multiple of 16

blockDim.x = 16  
blockDim.y = 16



```
float* dGPU = NULL;  
cudaMalloc((void**)&dGPU, n * n * sizeof(float));  
float* rGPU = NULL;  
cudaMalloc((void**)&rGPU, n * n * sizeof(float));  
cudaMemcpy(dGPU, d, n * n * sizeof(float),  
            cudaMemcpyHostToDevice);
```

**n/16, rounded up**

```
dim3 dimBlock(16, 16);  
dim3 dimGrid(divup(n, dimBlock.x), divup(n, dimBlock.y));  
mykernel<<dimGrid, dimBlock>>>(rGPU, dGPU, n);
```

```
cudaMemcpy(r, rGPU, n * n * sizeof(float),  
            cudaMemcpyDeviceToHost);  
cudaFree(dGPU); cudaFree(rGPU);
```

# Performance

- Test input:  $n = 6300$
- Maari computers:
  - baseline CPU solution: **397 s**
  - best CPU solution: **2.3 s**
  - current GPU solution: **42 s**
- What is the bottleneck?

# Memory

- A key challenge in CPU code:  
*getting data fast enough from the CPU memory*
- A key challenge in GPU code:  
*getting data fast enough from the GPU memory*

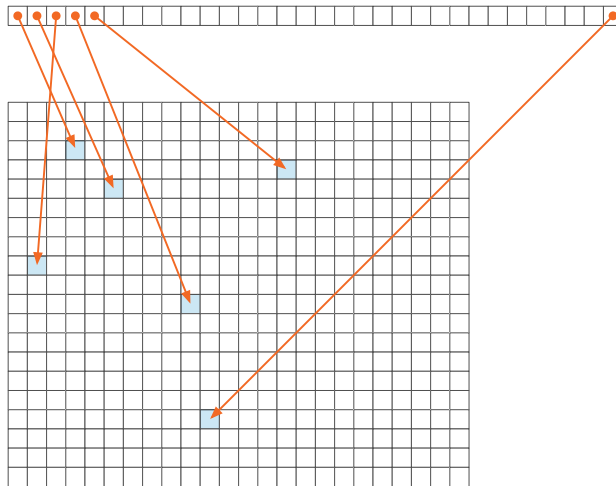


# Memory access pattern

- Blocks are divided in warps
  - warp = 32 threads
- *Entire warp executes synchronously*
- If one thread reads some memory,  
all threads of the warp read some memory

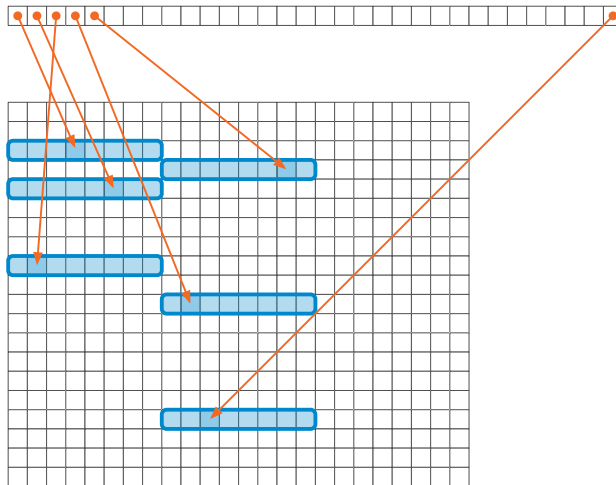
warp of threads:

GPU memory:



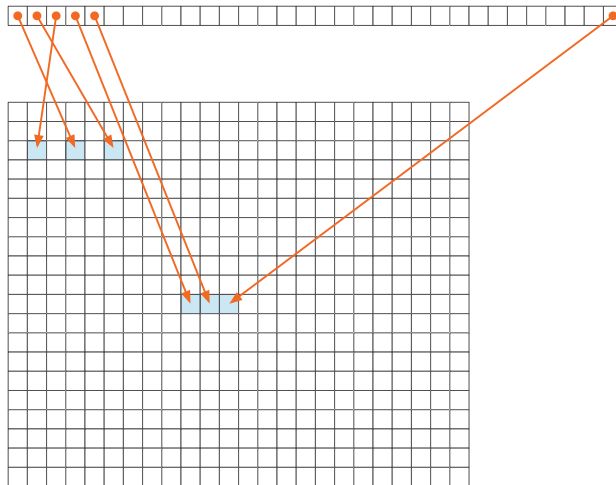
warp of threads:

GPU memory:



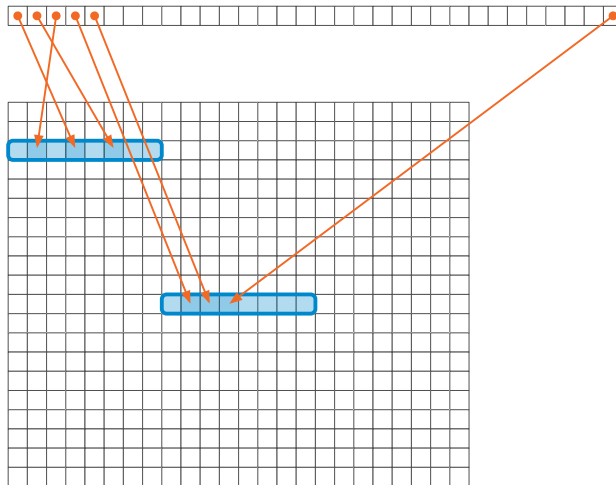
warp of threads:

GPU memory:



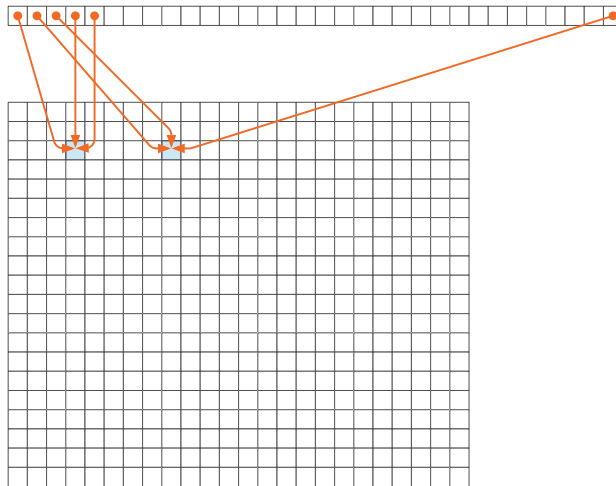
warp of threads:

GPU memory:



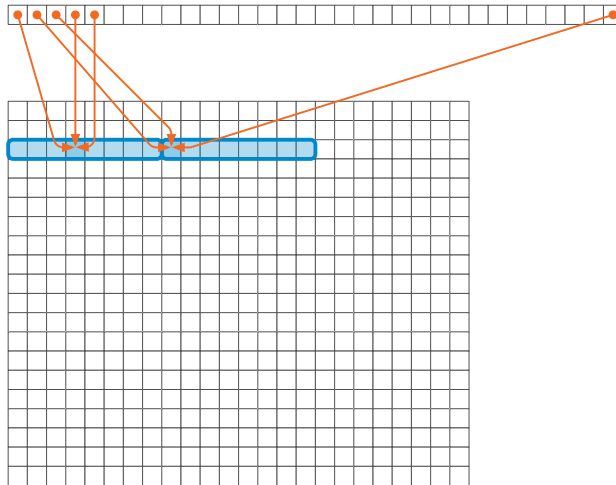
warp of threads:

GPU memory:



warp of threads:

GPU memory:



# Memory access pattern

- One memory read in kernel:  
*entire warp of threads reads memory simultaneously*
- Threads access small continuous parts of memory:  
need to load few cache lines → **good**
- Threads access 32 different locations far from each other:  
need to load many cache lines → **bad**



First warp:

thread 0: **i = 0**, **j = 0**

thread 1: **i = 1**, **j = 0**

thread 2: **i = 2**, **j = 0**

thread 3: **i = 3**, **j = 0**

...

thread 31: **i = 15**, **j = 1**

**Pay attention  
to this index!**

```
int i = threadIdx.x + ...  
int j = threadIdx.y + ...  
for (... ++k) {  
    float x = d[n*i + k];  
    float y = d[n*k + j];  
    ...  
}
```

Bad

First warp, first iteration:

threads 0 & 16: read `d[0]`

threads 1 & 17: read `d[1000]`

threads 2 & 18: read `d[2000]`

threads 3 & 19: read `d[3000]`

...

```
int i = threadIdx.x + ...  
int j = threadIdx.y + ...  
for (... ++k) {  
    float x = d[n*i + k];  
    float y = d[n*k + j];  
    ...  
}
```

Bad

First warp, first iteration:

threads 0 & 16: read **d[0]**

threads 1 & 17: read **d[1000]**

threads 2 & 18: read **d[2000]**

threads 3 & 19: read **d[3000]**

...

```
int i = threadIdx.x + ...
```

```
int j = threadIdx.y + ...
```

```
for (... ++k) {
```

```
    float x = d[n*i + k];
```

```
    float y = d[n*k + j];
```

```
    ...
```

```
}
```

Good

threads 0–15: read **d[0]**

threads 16–31: read **d[1]**

Bad

First warp, second iteration:

threads 0 & 16: read **d[1]**

threads 1 & 17: read **d[1001]**

threads 2 & 18: read **d[2001]**

threads 3 & 19: read **d[3001]**

...

```
int i = threadIdx.x + ...
```

```
int j = threadIdx.y + ...
```

```
for (... ++k) {
```

```
float x = d[n*i + k];
```

```
float y = d[n*k + j];
```

```
...
```

```
}
```

Good

threads 0–15: read **d[1000]**

threads 16–31: read **d[1001]**

Bad

First warp, third iteration:

threads 0 & 16: read **d[2]**

threads 1 & 17: read **d[1002]**

threads 2 & 18: read **d[2002]**

threads 3 & 19: read **d[3002]**

...

```
int i = threadIdx.x + ...
```

```
int j = threadIdx.y + ...
```

```
for (... ++k) {
```

```
    float x = d[n*i + k];
```

```
    float y = d[n*k + j];
```

```
    ...
```

```
}
```

Good

threads 0–15: read **d[2000]**

threads 16–31: read **d[2001]**

Exchange the  
roles of i and j

```
int i = threadIdx.x + ...  
int j = threadIdx.y + ...  
for (... ++k) {  
    float x = d[n*i + k];  
    float y = d[n*k + j];  
    float x = d[n*j + k];  
    float y = d[n*k + i];  
    ...  
}
```

```
r[n*i + j] = v;  
r[n*j + i] = v;
```

Good

First warp, first iteration:

```
int i = threadIdx.x + ...
int j = threadIdx.y + ...
for (... ++k) {
  threads 0–15: read d[0]      float x = d[n*j + k];
  threads 16–31: read d[1000] float y = d[n*k + i];
  ...
}
```

First warp, first iteration:

```
int i = threadIdx.x + ...  
int j = threadIdx.y + ...  
for (... ++k) {  
    float x = d[n*j + k];  
    float y = d[n*k + i];  
    ...  
}
```

Good

threads 0–15: read **d[0]** ————— **float x = d[n\*j + k];**  
threads 16–31: read **d[1000]** **float y = d[n\*k + i];**  
...

Good

threads 0 & 16: read **d[0]**  
threads 1 & 17: read **d[1]**  
threads 2 & 18: read **d[2]**  
...



First warp, second iteration:

```
int i = threadIdx.x + ...  
int j = threadIdx.y + ...  
for (... ++k) {  
    float x = d[n*j + k];  
    float y = d[n*k + i];  
    ...  
}
```

Good

threads 0–15: read **d[1]** ————— **float x = d[n\*j + k];**  
threads 16–31: read **d[1001]** **float y = d[n\*k + i];**

Good

threads 0 & 16: read **d[1000]**  
threads 1 & 17: read **d[1001]**  
threads 2 & 18: read **d[1002]**  
...

First warp, third iteration:

```
int i = threadIdx.x + ...  
int j = threadIdx.y + ...  
for (... ++k) {  
    float x = d[n*j + k];  
    float y = d[n*k + i];  
    ...  
}
```

Good

threads 0–15: read **d[2]** ————— **float x = d[n\*j + k];**  
threads 16–31: read **d[1002]** **float y = d[n\*k + i];**

Good

threads 0 & 16: read **d[2000]**  
threads 1 & 17: read **d[2001]**  
threads 2 & 18: read **d[2002]**  
...

# Performance

- **V0**: baseline — **42 s**
- **V1**: better memory access pattern — **8 s**
- But we can do much better by applying familiar ideas:
  - **reuse data in registers**
  - **reuse data in “cache”** (here: shared memory)

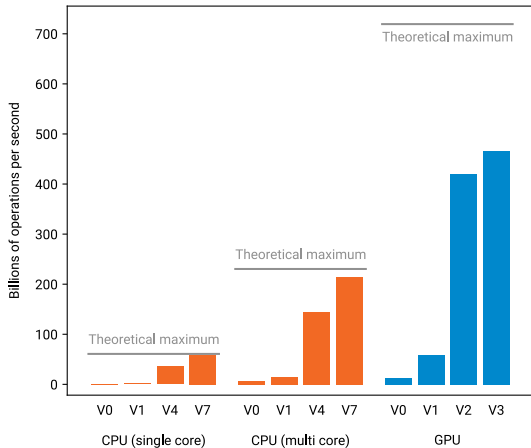
# Performance

**V0:** baseline

**V1:** memory access

**V2:** registers

**V3:** shared memory



# Plan

- 1 GPU programming
- 2 GPU programming with CUDA
- 3 Sample application: shortcut problem
- 4 Memory access patterns in CUDA programs
- 5 Warps, blocks, and shared memory**
- 6 How does the GPU execute code?
- 7 GPU programming — conclusions

# Warps and blocks

- Threads in GPUs are organized in two ways:
  - **warps** (always **32** threads)
  - **blocks** (you can choose the number of threads)
- Why do we need these concepts?
  - warps: help the **hardware**
  - blocks: help the **programmer**

# What if there were no warps?

- Our GPU has 640 arithmetic units for doing scalar operations
- If we had only individual threads, you would need **640 schedulers** that process instructions from individual threads and move operands to the right arithmetic units
- By organizing threads in warps, we only need **20 schedulers** that process instructions from complete warps and move warp-wide operands to warp-wide arithmetic units
- Less space & energy used by control logic, more space & energy left for useful work

# What if there were no warps?

- Similar to CPUs & vector operations:
  - make *arithmetic units and registers wider* by a factor of 8:  
more processing power without adding much more control logic
  - *increase the number of cores* by a factor of 8:  
everything got 8 times more costly



# Blocks and shared memory

- Blocks are there to help you!
- You can allocate a small amount of very fast **“shared memory”** for each block
  - “small amount”  $\approx$  kilobytes per block
  - “very fast”  $\approx$  L1 cache
- All threads of a block see the same shared memory
- Threads of a block can use shared memory to **communicate with each other** and coordinate their work

# Blocks and shared memory

- **Example:** each block calculates a sum using many threads
  - $b$  threads per block
  - allocate  $b$  words of shared memory per block
  - split input in  $b$  parts
  - thread  $i$  calculates a local sum in its own part and **writes** it in element  $i$  in shared memory
  - **synchronization:** wait all threads to finish writing
  - thread 0 **reads** all local sums from shared memory and calculates the grand total

# Warps and blocks

- **Warps:**

- always 32 threads
- **helps with hardware design:**  
**lots of arithmetic power with a simpler control**
- you will have to live with this even if it is inconvenient for you

- **Blocks:**

- you can choose the block size (e.g. 64 or 256 threads)
- **threads of a block can easily and efficiently communicate with each other using “shared memory”**
- blocks are a useful feature that you can use

# Using shared memory in CUDA

```
__global__ void mykernel() {  
    __shared__ float x[100];  
    ...  
}
```

One array  
per block!

Different:

x[0] in block 10

x[0] in block 11

Same:

x[0] in thread 5 of block 10

x[0] in thread 6 of block 10

# Using shared memory in CUDA

```
__global__ void mykernel() {  
    __shared__ float x[100];  
    ...  
    __syncthreads();  
    ...  
}
```

A thread won't continue until all threads of the block have reached this point

# Using shared memory in CUDA

...

Write to my own slot

`x[i] = a;`

# Using shared memory in CUDA

...

Write to my own slot

`x[i] = a;`

Wait for everyone to finish writing

`__syncthreads();`

# Using shared memory in CUDA

...

Write to my own slot

`x[i] = a;`

Wait for everyone to finish writing

`__syncthreads();`

`b = x[0];`

Now safe to read from any slot



# Using shared memory in CUDA

...

Write to my own slot

`x[i] = a;`

Wait for everyone to finish writing

`__syncthreads();`

`b = x[0];`

Now safe to read from any slot

`__syncthreads();`

Wait for everyone to finish reading

# Using shared memory in CUDA

...

Write to my own slot

`x[i] = a;`

Wait for everyone to finish writing

`__syncthreads();`

`b = x[0];`

Now safe to read from any slot

`__syncthreads();`

Wait for everyone to finish reading

`x[i] = c;`

Now safe to write again

...

# Shared memory is small

- **64 KB** in total per SM (“streaming multiprocessor”)
- Example:
  - you want to have 8 active blocks on each SM
  - you can only allocate at most 8 KB of shared memory per block

# Key elements of CUDA programs

- Allocating **GPU memory**, moving data between CPU memory and GPU memory
- Creating **blocks of threads**, launching **kernels**
- Allocating **shared memory** for sharing data inside a block, using `__syncthreads` to synchronize work

# Plan

- 1 GPU programming
- 2 GPU programming with CUDA
- 3 Sample application: shortcut problem
- 4 Memory access patterns in CUDA programs
- 5 Warps, blocks, and shared memory
- 6 How does the GPU execute code?
- 7 GPU programming — conclusions

# What happens inside the GPU?

- *The same general principles hold for a wide range of different GPUs*
- However, when we need some concrete numbers to illustrate these ideas, we will use the following GPU:
  - NVIDIA Quadro K2200
  - “**Maxwell**” microarchitecture
  - **5 × streaming multiprocessors** (SM)

# Key concepts that we need

- Kernel  $\approx$  some instructions that we want to execute
- Blocks that consist of warps
- Warps that consist of 32 threads
- Shared memory
- **Registers**

# GPU registers

Lots of data,  
needs to be stored  
somewhere!

- **At most 255 registers** per thread
  - scalar registers, can hold 32-bit numbers
- When your kernel is compiled,  
**the compiler will decide how many registers are used**
  - for each kernel, the compiler stores some metadata, e.g.:

"To run this kernel,  
we will need 96 registers per thread,  
and 2 KB of shared memory per block"



```
__global__ void mykernel(...) {  
    ...  
    float v[8][8];  
    ...  
    for (int k = 0; k < n; ++k) {  
        float x[8];  
        float y[8];  
        ...  
        x[ib] = ...;  
        y[jb] = ...;  
        ...  
        v[ib][jb] = min(v[ib][jb], x[ib] + y[jb]);  
    }  
    ...  
}
```

64 registers?

8 + 8 registers?

## cuobjdump --dump-sass

```
...  
FADD R79, R86.reuse, R79 ;  
FADD R85, R86.reuse, R85 ;  
FADD R89, R86, R89 ;  
FMNMX R69, R88, R69, PT ;  
FMNMX R67, R90, R67, PT ;  
FMNMX R56, R75, R56, PT ;  
FMNMX R53, R95, R53, PT ;  
FMNMX R34, R87, R34, PT ;  
FMNMX R26, R83, R26, PT ;  
...
```

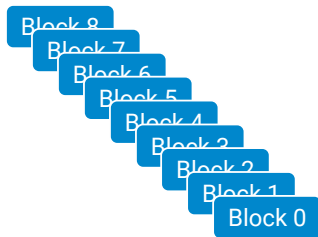
**Uses 96 registers  
(R0 ... R95)**

# Key choices

- **Fixed:** 32 *threads per warp*
- **We choose:** how many *threads per block*
  - at most 1024
- **We choose:** how much *shared memory per block*
  - at most 48 KB
- **Compiler chooses:** how many *registers per thread*
  - depends on our kernel code
  - at most 255

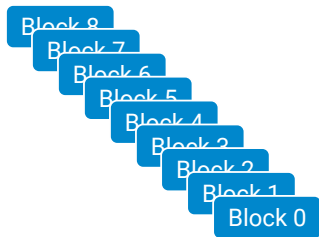
# What happens when we launch a kernel?

- All **blocks** are put in a GPU-wide queue
  - cheap, no resources allocated yet



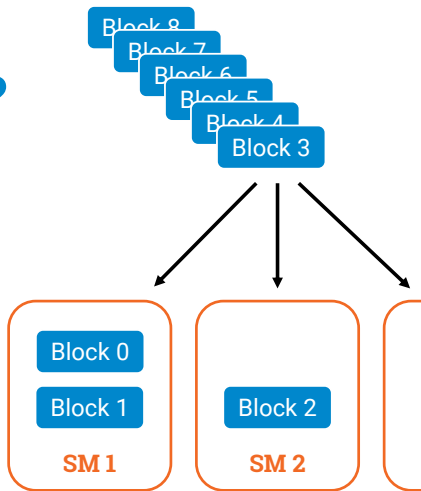
# What happens when we launch a kernel?

- All **blocks** are put in a GPU-wide queue
  - cheap, no resources allocated yet
- 5 “*streaming multiprocessors*” (SM)



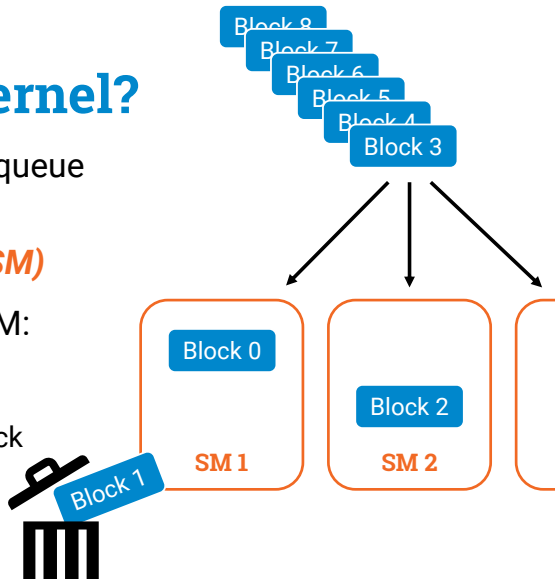
# What happens when we launch a kernel?

- All **blocks** are put in a GPU-wide queue
  - cheap, no resources allocated yet
- **5 “streaming multiprocessors” (SM)**
- Whenever there is room in one SM:
  - SM takes a block from the queue
  - the block becomes active
  - resources are allocated for the block



# What happens when we launch a kernel?

- All **blocks** are put in a GPU-wide queue
  - cheap, no resources allocated yet
- **5 “streaming multiprocessors” (SM)**
- Whenever there is room in one SM:
  - SM takes a block from the queue
  - the block becomes active
  - resources are allocated for the block
  - the block is there until all threads in the block finish running, then resources are freed



# What happens when SM starts to process a block?

- Block becomes active
  - room for **32 active blocks** per SM
- All warps of the block become active
  - room for **64 active warps** per SM
- Shared memory allocated for the block
  - **64 KB shared memory** available per SM
- Physical registers allocated for each thread
  - **65536 physical 32-bit registers** per SM

**Blocks will have to wait in the queue until all these resources are available!**



# What happens when SM starts to process a block?

- Block becomes active
  - room for **32 active blocks** per SM
- All warps of the block become active
  - room for **64 active warps** per SM
- Shared memory allocated for the block
  - **64 KB shared memory** available per SM
- Physical registers allocated for each thread
  - **65536 physical 32-bit registers** per SM

**64 active warps**  
**× 32 threads/warp**  
**× 5 SMs**  
**= 10240 active threads**

# What happens when SM starts to process a block?

- Block becomes active
  - room for **32 active blocks** per SM
- All warps of the block become active
  - room for **64 active warps** per SM
- Shared memory allocated for the block
  - **64 KB shared memory** available per SM
- Physical registers allocated for each thread
  - **65536 physical 32-bit registers** per SM

**Limits parallelism  
if blocks too small**

# What happens when SM starts to process a block?

- Block becomes active
  - room for **32 active blocks** per SM
- All warps of the block become active
  - room for **64 active warps** per SM
- Shared memory allocated for the block
  - **64 KB shared memory** available per SM
- Physical registers allocated for each thread
  - **65536 physical 32-bit registers** per SM

**64 warps**  
**× 32 threads/warp**  
**× 32 registers/thread**  
**= 65536 registers**

# What happens when SM starts to process a block?

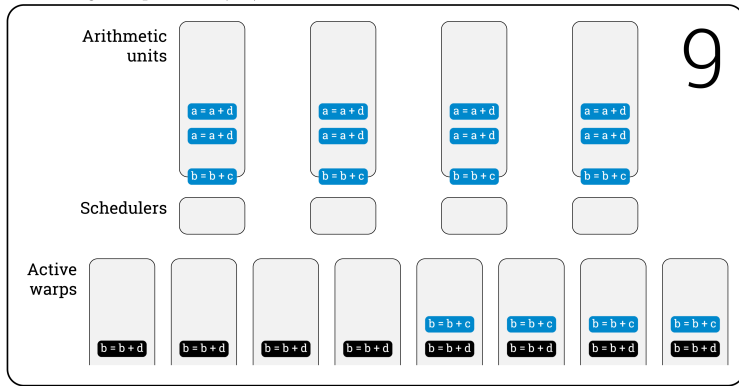
- Block becomes active
  - room for **32 active blocks** per SM
- All warps of the block become active
  - room for **64 active warps** per SM
- Shared memory allocated for the block
  - **64 KB shared memory** available per SM
- Physical registers allocated for each thread
  - **65536 physical 32-bit registers** per SM

**32 active blocks:  
2 KB shared  
memory per block**

# How does SM execute code from active warps?

# See the videos for an animation...

Streaming multiprocessor (SM)



# Keeping arithmetic units busy (in theory)

- **Lots of independent instructions:**
  - e.g. floating-point additions: throughput **4 warps per clock cycle**
  - **4 active warps** per SM enough to keep all arithmetic units busy
  - in each clock cycle there is something to do in each warp
- **All instructions depend on previous instruction:**
  - e.g. floating-point addition: latency **6 clock cycles**
  - **$6 \cdot 4 = 24$  active warps** per SM enough to keep arithmetic units busy
  - in each clock cycle there is a warp that is ready

# Keeping arithmetic units busy (in theory)

- **Lots of independent instructions:**

- e.g. floating-point additions: throughput **4 warps per clock cycle**
- **4 active warps** per SM enough to keep arithmetic units busy
- in each clock cycle there is a warp ready to execute in each warp

The real hardware is  
a bit more complicated...

- **All instructions are independent instructions:**

- e.g. floating-point additions: latency **6 clock cycles**
- **$6 \cdot 4 = 24$  active warps** per SM enough to keep arithmetic units busy
- in each clock cycle there is a warp that is ready



# Keeping arithmetic units busy (in practice)

- **Lots of independent “+” instructions:**
  - **4 active warps** per SM enough to keep arithmetic units  $\geq$  **82%** busy
  - **8 active warps** per SM enough to keep arithmetic units  $\geq$  **96%** busy
- **Pairs of independent “+” instructions:**
  - **12 active warps** per SM enough to keep arithmetic units  $\geq$  **87%** busy
  - **16 active warps** per SM enough to keep arithmetic units  $\geq$  **97%** busy
- **All “+” instructions depend on previous instruction:**
  - **16 active warps** per SM enough to keep arithmetic units  $\geq$  **65%** busy
  - *additional warps do not help to get beyond 65%*

# Plan

- 1 GPU programming
- 2 GPU programming with CUDA
- 3 Sample application: shortcut problem
- 4 Memory access patterns in CUDA programs
- 5 Warps, blocks, and shared memory
- 6 How does the GPU execute code?
- 7 GPU programming — conclusions**

# What if the threads of a warp try to do different things?

- How are these two claims compatible:
  - all threads of a warp work in a synchronous manner
  - kernel is arbitrary C++ code written from the perspective of a thread
- *Then what happens if different threads of a warp try to do different things?*
- For example, what if different threads have different values of x:
  - `if (x < 123) { ... } else { ... }`
  - `for (int i = 0; i < x; ++i) { ... }`

# What if the threads of a warp try to do different things?

...

```
if (x < 123) {
```

True for threads 0...15

```
    ...
```

```
} else {
```

Entire warp takes these steps,  
but threads 16...31 are disabled

```
    ...
```

```
}
```

Entire warp takes also these steps,  
but threads 0...15 are disabled

...

# What if the threads of a warp try to do different things?

- You *can* write arbitrary C++ code in which different threads do completely different things, it will be executed correctly!
- But it may be very inefficient, e.g.:
  1. the warp follows what **thread 0** does (threads 1, 2, 3, ..., 31 disabled)
  2. the warp follows what **thread 1** does (threads 0, 2, 3, ..., 31 disabled)
  3. ...
- *You can lose in performance by a factor of 32 if you don't keep in mind that the entire warp is executed synchronously*

# Compilation process and GPU assembly language

- C++ → PTX → SASS
  - PTX: platform independent intermediate language
  - SASS: what the GPU runs
- You can use **cuobjdump --dump-sass** to show the SASS code

# Block-wide vs. warp-wide communication

- **Communication between the threads of a block:**
  - allocate shared memory with `__shared__`
  - read/write shared memory
  - synchronize with `__syncthreads()`
- **Communication between the threads of a warp:**
  - call e.g. functions `__shfl_sync()` or `__shfl()`
  - see *CUDA C++ Programming Guide*

# GPU programming recap

- **You need to explicitly say what the GPU should run**
  - write a **kernel**, specify how many **blocks** of threads you want, specify how many **threads** there are per block, launch the kernel
- **All threads will run the same kernel code**
  - in the kernel you can use the **thread index** and **block index** to decide what to do
- **GPU-side code accesses only GPU memory**
  - you need to use CUDA functions to move data between CPU memory and GPU memory



# GPU programming recap

- **Threads are organized in *warps* of 32 threads**
  - all threads of a warp are always synchronized
  - pay attention to memory access pattern
- **Threads are organized in *blocks* of  $x$  threads**
  - threads of a block can use shared memory for communication
- **GPU executes instructions in a linear order**
  - only looks at the next instruction in each active warp
  - *good to have lots of active warps*
  - number of active warps limited by register & shared memory usage

# What you learned earlier still applies

- It is always a good idea to try to **minimize memory reads** by reusing data in registers
- The same idea works both on CPUs and on GPUs
- See the course material for examples!